

高层运动服务接口

来源：https://support.unitree.com/home/zh/A2_SDK_Development_Guide/sport_client_interface

高层运动服务接口

更新时间：2026-04-24 21:18:48

高层运动服务接口

高层运动服务接口的主要功能为机器人整机动作控制。

注意：

高层运动服务依赖于内置运控，进入调试模式后内置运控完全退出，高层运动服务失效。

介绍

高层运动服务分为两个部分：高层控制接口、高层状态接口。

- **高层控制接口**：通过调用 SDK 的 sport_client，来给 A2 发送速度控制、轨迹跟踪等运动指令。
- **高层状态接口**：通过订阅 SDK 中的 HigState 消息，来获取 A2 的位置、速度、姿态等运动状态 (unitree_go::msg::dds_::SportModeState_)。

高层运动控制接口介绍

高层控制接口可以实现 A2 的姿态、速度空翻、跳跃、倒立等控制。下面列出运动控制接口的相关函数。

函数名	Damp
函数原型	int32_t Damp()
功能概述	进入阻尼状态。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	所有电机关节停止运动并进入阻尼状态。该模式具有最高的优先级，用于突发情况下的急停。

函数名	BalanceStand
函数原型	int32_t BalanceStand()
功能概述	解除锁定。
参数	无

函数名	BalanceStand
返回值	调用成功返回 0，否则返回相关错误码。
备注	解除关节电机锁定，从正常站立、趴下、持续踏步状态切换到平衡站立模式。该模式下，拨动遥控器摇杆，机器人会移动。

函数名	StopMove
函数原型	int32_t StopMove()
功能概述	停下当前动作，将绝大多数指令恢复成默认值。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	停下当前的运动，并将 A2 内部的运动参数恢复到默认值。

函数名	StandUp
函数原型	int32_t StandUp()
功能概述	关节锁定，站高。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	机器狗正常站高，电机关节保持锁定。相比于平衡站立模式，该模式下机器狗的姿态不会始终保持平衡。

函数名	StandDown
函数原型	int32_t StandDown()
功能概述	关节锁定，站低。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	机器狗趴下，电机关节保持锁定。

函数名	RecoveryStand
函数原型	int32_t RecoveryStand()
功能概述	恢复站立。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	从翻倒状态恢复至平衡站立状态。为了安全，只会在阻尼/趴下状态下，才会响应恢复至站立。

函数名	Move																
函数原型	int32_t Move(float vx, float vy, float vyaw)																
功能概述	移动。																
参数	参数 ● vx： 机体坐标系x速度 ● vy： 机体坐标系y速度 ● vyaw： 机体坐标系yaw速度 <table><tr><th>模式</th><th>档位</th><th>速度</th></tr><tr><td rowspan="2">走路</td><td>低速</td><td>[±0.8,±0.5,±2.0]</td></tr><tr><td>高速</td><td>[±1.5,±0.5,±2.5]</td></tr><tr><td rowspan="2">跑步</td><td>低速</td><td>[±2.5,±0.3,±1.5]</td></tr><tr><td>高速</td><td>[±4.0,±0.3,±1.0]</td></tr><tr><td>攀爬模式</td><td>/</td><td>[±0.6,±0.3,±1.0]</td></tr></table>	模式	档位	速度	走路	低速	[±0.8,±0.5,±2.0]	高速	[±1.5,±0.5,±2.5]	跑步	低速	[±2.5,±0.3,±1.5]	高速	[±4.0,±0.3,±1.0]	攀爬模式	/	[±0.6,±0.3,±1.0]
模式	档位	速度															
走路	低速	[±0.8,±0.5,±2.0]															
	高速	[±1.5,±0.5,±2.5]															
跑步	低速	[±2.5,±0.3,±1.5]															
	高速	[±4.0,±0.3,±1.0]															
攀爬模式	/	[±0.6,±0.3,±1.0]															
返回值	调用成功返回 0，否则返回相关错误码。																
备注	走路、跑步和攀爬模式可以通过 SwitchGait 接口切换，速度档位的切换可以通过 SpeedLevel 接口实现。																

函数名	BodyHeight
函数原型	int32_t BodyHeight(float height)
功能概述	设置站立和行走时的机身离地高度相对值。
参数	height： 与默认机身高度相对值，取值[0.3~0.5] (m)
返回值	调用成功返回 0，否则返回相关错误码。
备注	调节平衡站立或行走时的机身相对与默认状态的高度。

函数名	Euler
函数原型	int32_t Euler(float roll, float pitch, float yaw)
功能概述	摆姿势
参数	roll：取值范围[-0.6~0.6],pitch：取值范围[-0.6~0.6],yaw：取值范围[-0.6~0.6]
返回值	调用成功返回 0，否则返回相关错误码。
备注	调用Euler之前，建议先调用一次BalanceStand。

函数名	SwitchGait
函数原型	int32_t SwitchGait(int d)
功能概述	切换步态。

函数名	SwitchGait
参数	d ：步态枚举值，取值 0~2，0 为 默认行走，1 为 跑步模式，2 为 攀爬模式。
返回值	调用成功返回 0，否则返回相关错误码。
备注	为了安全，步态切换需要在可移动状态才生效。调用SwitchGait前，需先调用一次BalanceStand。

函数名	SpeedLevel
函数原型	int32_t SpeedLevel(int level)
功能概述	设置速度档位。
参数	level ：速度档位枚举值，取值 0 为低速，1 为高速。
返回值	调用成功返回 0，否则返回相关错误码。
备注	

函数名	SetAutoRecover
函数原型	int32_t SetAutoRecovery(int switch_on)
功能概述	设置自动翻身是否生效。
参数	int：设置0，自动翻身失效；设置1，自动翻身生效
返回值	调用成功返回 0，否则返回相关错误码。
备注	当设备有背载时，建议您关闭该配置。因为如果设备摔倒并剧烈翻转，可能会损坏背载设备，像机身头部安装的云台、相机等传感器等都容易受到影响。为了避免这种情况，请您务必注意关闭该配置。

函数名	GetState
函数原型	int32_t GetState(std::map<std::string, std::string> &state_map)
功能概述	获取当前A2的状态机ID、状态机名、速度挡位、自动翻身设置与是否正在做动作
参数	state_map：包括： fsm_id：状态机ID fsm_name：状态机名称 speed_level：速度挡位 auto_recovery_switch：自动翻身开关 process_state：是否正在做动作
返回值	调用成功返回 0，否则返回相关错误码。
备注	

GetState接口使用方法示例

获取当前fsm_id

```

#include <cmath>
#include <unitree/robot/channel/channel_subscriber.hpp>
#include <unitree/idl/go2/SportModeState_.hpp>

#define TOPIC_HIGHSTATE "rt/lf/sportmodestate"
using namespace unitree::common;

const std::vector<std::string> FSM_STATE_STR = {
    "PASSIVE",
    "STAND_DOWN",
    "STAND_UP",
    "DEFAULT_MODE",
    "RUNNING_MODE",
    "CLIMB_MODE",
    "LEFT_SIDE_GAIT",
    "RIGHT_SIDE_GAIT",
    "HANDSTAND",
    "BIPED_STAND",
    "FRONT_FLIP",
    "BACK_FLIP",
    "RECOVERY",
    "BASE_HEIGHT_CTRL"};

class Custom
{
public:
    Custom()
    {
        suber.reset(new
unitree::robot::ChannelSubscriber<unitree_go::msg::dds_::SportModeState_>
(TOPIC_HIGHSTATE));
        suber->InitChannel(std::bind(&Custom::HighStateHandler, this,
std::placeholders::_1), 1);
    };

    void HighStateHandler(const void *message)
    {
        state = *(unitree_go::msg::dds_::SportModeState_ *)message;

        std::cout << "Position: " << state.position()[0] << ", " << state.position()[1]
<< ", " << state.position()[2] << std::endl;
        std::cout << "Velocity: " << state.velocity()[0] << ", " << state.velocity()[1]
<< ", " << state.velocity()[2] << std::endl;
    }
};

```

```

std::cout << "Mode: " << FSM_STATE_STR[int(state.mode())] << std::endl;
std::cout << "Progress: " << state.progress() << std::endl;

// std::cout << "IMU rpy: " << state.imu_state().rpy()[0] << ", " <<
state.imu_state().rpy()[1] << ", " << state.imu_state().rpy()[2] << std::endl;
};

unitree_go::msg::dds_::SportModeState_ state;
unitree::robot::ChannelSubscriberPtr<unitree_go::msg::dds_::SportModeState_> suber;
float dt = 0.05; // 控制步长0.001~0.01
};

int main(int argc, char **argv)
{
    if (argc < 2)
    {
        std::cout << "Usage: " << argv[0] << " networkInterface" << std::endl;
        exit(-1);
    }

    unitree::robot::ChannelFactory::Instance()->Init(0, argv[1]);
    Custom custom;

    while (1)
    {
        sleep(10);
    }
    return 0;
}

```

接口错误码如下：

错误码	错误描述
4201	动作超时错误，在期望的时间内没有完成指定动作
4205	状态机未初始化结束
3104	DDS 超时

高层状态接口

高层状态接口调用方式

通过订阅rt/sportmodestate话题可以实现对机器人当前的里程计信息、速度、当前模式、是否正在执行动作。

运控模式的状态机ID和说明

ID	运控名称	备注
0	PASSIVE	阻尼模式
1	STAND_DOWN	站低
2	STAND_UP	站高
3	DEFAULT_MODE	默认步态
4	RUNNING_MODE	跑步模式
5	CLIMB_MODE	攀爬模式
6	LEFT_SIDE_GAIT	左侧步
7	RIGHT_SIDE_GAIT	右侧步
8	HANDSTAND	倒立
9	BIPED_STAND	直立
10	FRONT_FLIP	前空翻
11	BACK_FLIP	后空翻
12	RECOVERY	翻身
13	BASE_HEIGHT_CTRL	机身高度控制步态

高层状态类型介绍

状态名	IMU State
定义	const ::unitree_go::msg::dds_::IMUState_& imu_state
内容	IMU数据
备注	具体数据类型和内容可参考 底层服务接口

状态名	Mode
定义	uint8_t mode
内容	当前运控模式状态机ID

状态名	Mode
备注	具体ID见上文运控模式的状态机ID和说明

状态名	Position
定义	const std::array<float, 3>& position
内容	里程计数据, xyz
备注	

状态名	Velocity
定义	const std::array<float, 3>& velocity
内容	速度数据, xyz
备注	

状态名	Progress
定义	float progress_;
内容	是否正在执行动作
备注	